

INSTITUTE OF ELECTRONIC SYSTEMS

TTT4275 - ESTIMATION, DETECTION AND CLASSIFICATION

Classification project

Authors :
Arta Osmani, Karin Skinderhaug

April, 2025

Summary

This project explores the design, training, and evaluation of classifiers for two classification tasks: the Iris flower dataset and the pronounced vowels dataset.

For the Iris dataset, a linear classifier was trained using batch gradient descent and mean squared error (MSE) as the loss function. The model was first evaluated using two different data splits, followed by an analysis of how individual features contributed to class separability. Results showed strong performance, with test error rates as low as 0% when using all features, increasing up to 8.33% as fewer features were used. This is to show that data-splitting is important as well as more information is preferred even though there is overlap.

The vowel classification task used generative models, including single Gaussian classifiers and Gaussian Mixture Models (GMMs). The models were tested using full- and diagonal covariance matrices. The best performance was achieved using a single Gaussian with full covariance (error rate 34.18%), while GMMs with diagonal covariance (two and three components) provided moderate improvements over the diagonal single Gaussian. The analysis also revealed that the class imbalance between the training and test sets, specifically differences in speaker demographics, negatively affected performance.

Overall, the project highlights the importance of proper model selection, feature analysis, and data splitting strategies in classification tasks. It also demonstrates the practical challenges and trade-offs involved in real-world data modeling

Table of Contents

List of Figures	ii
List of Tables	iii
1 Introduction	1
2 Theory	1
2.1 Linear classifier	1
2.2 Plug-in-MAP classifier	2
2.3 Model-fitting and generalization	2
2.4 Evaluation of classifiers	3
3 Our task	4
3.1 Iris task	4
3.2 Pronounced vowels task	4
4 Implementation and results	4
4.1 Iris task	4
4.2 Pronounced vowels task	7
5 Discussion	9
5.1 Classification of Iris flowers	9
5.2 Classification of pronounced vowels	10
6 Conclusion	11
Bibliography	12
Appendix	13
A Confusion matrix task 2	13

List of Figures

1	Visual model showing, underfitting, fitting and overfitting	3
2	MSE versus iterations for different values of the learning rate α . The purple curve, corresponding to $\alpha = 0.6$, performs best with the lowest MSE across iterations.	5
3	Confusion matrix for the training data using the first 30 samples for training and last 20 for testing. True classes are on the vertical axis; predicted classes on the horizontal.	6
4	Confusion matrix for the testing data using the first 30 samples for training and last 20 for testing. True classes are on the vertical axis; predicted classes on the horizontal.	6
5	Confusion matrix for the training data using the last 30 samples for training and first 20 for testing. True classes are on the vertical axis; predicted classes on the horizontal.	6
6	Confusion matrix for the testing data using the last 30 samples for training and first 20 for testing. True classes are on the vertical axis; predicted classes on the horizontal.	6
7	Histograms showing feature distributions across the classes; iris-setosa (pink), iris-versicolor (purple) and iris-virginica (orange).	7

8	Confusion matrix for the test data using a single Gaussian classifier with full covariance matrices. The true vowels are represented on the vertical axis, while the predicted vowels are displayed on the horizontal axis.	8
9	Confusion matrix for the test data using a single Gaussian classifier with diagonal covariance matrices. The true vowels are represented on the vertical axis, while the predicted vowels are displayed on the horizontal axis.	8
10	Confusion matrix for the test data using GMM-2 classifier with diagonal covariance matrices. The true vowels are represented on the vertical axis, while the predicted vowels are displayed on the horizontal axis.	9
11	Confusion matrix for the test data using a GMM-3 classifier with diagonal covariance matrices. The true vowels are represented on the vertical axis, while the predicted vowels are displayed on the horizontal axis.	9
12	Histogram of features F1 (pink), F2 (purple), and F3 (yellow) for vowels "oo" and "er" in both the training and test sets.	10
13	Confusion matrices for training and test including different amount of features . . .	13

List of Tables

1	Confusion matrix (theoretical example with three classes)	3
2	Classification error for different feature subsets and data splits	7

1 Introduction

The goal of this project is to explore the design, training, and evaluation classifiers, applied to two classification tasks: Iris flower classification and vowel sound classification. The project is part of the TTT4175 course in Estimation, Detection, and Classification, which emphasizes the practical implementation of classification systems.

Using artificial intelligence to train classifiers has had a lot of focus in recent years. This report presents the theoretical background needed to understand the tasks, followed by our implementation approach and results for both classification problems. We conclude by summarizing our findings and reflecting on the performance and behavior of the classifiers.

2 Theory

To understand how classifiers work, it is important to first introduce the basic concepts behind classification. A classifier is a model that assigns input data to predefined categories based on patterns it has learned from training data. In this section, we will cover key ideas such as decision boundaries, linear vs. non-linear classifiers, and how classifiers learn from data. This section introduces the theory necessary to understand the methods used in the project.

2.1 Linear classifier

Classification is a fundamental problem in machine learning, where the goal is to correctly assign an input vector to one of several predefined classes. A key question in such problems is whether the data is linearly separable, that is, if a straight line in 2D or a hyperplane in higher dimensions can perfectly divide the classes. If not, the data is non-linearly separable [1, p. 6].

Classification models are typically either generative or discriminative. Generative models learn the underlying distribution of each class, while discriminative models focus on finding decision boundaries between classes. A common example of a discriminative model is the linear classifier [1, p. 6].

To train classification models, we use a loss function, also called a cost function, to measure how far the predicted output is from the true class label. This function provides feedback on the performance of the model, guiding it to gradually adjust its parameters to minimize the error. A commonly used loss function is the mean squared error (MSE), defined in equation (1) [1, p. 16].

$$\text{MSE}(g, t) = \frac{1}{2} \sum_{k=1}^N (g_k - t_k)^T (g_k - t_k) \quad (1)$$

Here, g_{ic} is the output of the model for sample i and class c , and t_{ic} is the corresponding target value from a one-hot encoded vector.

The output g is computed from the input vector $x \in \mathbb{R}^D$ as follows:

$$g = \sigma(Wx) \quad (2)$$

x is an input vector with D features. The weight matrix $W \in \mathbb{R}^{C \times D}$ contains the parameters that the model learns during training. It transforms the input into a set of C outputs, where C is the number of classes [1, p. 10]. The function $\sigma(z)$ is the sigmoid activation function:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (3)$$

The sigmoid function introduces smoothness and non-linearity, which helps during optimization and allows the model to output values between 0 and 1, suitable for interpreting as probabilities [1, p. 16].

Finally, the training is done using gradient descent, an iterative algorithm that updates the weights W in the direction that reduces the MSE [1, p. 16]. This theoretical framework provides a foundation for understanding how the classifier works and why it is suitable for the Iris dataset.

2.2 Plug-in-MAP classifier

In this section, the theory about the Plug-in-MAP classifier is presented. This is also called a Bayes Decision rule (BDR) classifier since it is based on BDR. The equation for BDR is presented in (4) [1, p. 7].

$$x \in \omega_j \Leftrightarrow p(x|\omega_j)P(\omega_j) = \max_i p(x|\omega_i)P(\omega_i) \quad (4)$$

This is the optimal classification rule, assigning x to the class ω_j that maximizes the product of the likelihood $p(x | \omega_i)$ and the prior $P(\omega_i)$. However, using this equation requires a perfect density function describing the feature distribution for each class, which is rarely available in real-world scenarios. Instead, the solution is to approximate the class distribution using parameterized models that can be estimated from a dataset [1, p. 7].

One of the densities that can be used is the Gaussian density. The parameters that need to be calculated in this density is the mean μ and covariance Σ . For a single Gaussian density is the equation (5) [1, p. 9].

$$p(x|\omega_i) = \frac{1}{\sqrt{(2\pi)^D |\Sigma_i|}} \exp \left[-\frac{1}{2} (x - \mu_i)^\top \Sigma_i^{-1} (x - \mu_i) \right], \quad i = 1, \dots, C \quad (5)$$

By using maximum likelihood on the equation, we will find that the mean μ and covariance Σ should be estimated as the sample mean and the sample covariance matrix [1, p. 14].

Often, especially when each class has several features, is the real distribution of the classifier noisy and have several peaks. A single Gaussian classifier is not suitable in this case. Instead, a mixture of multiple Gaussian distributions can be used. By combining M Gaussian components, we obtain a Gaussian Mixture Model (GMM), which represents a probabilistic model where each data point is assumed to be generated from one of several Gaussian distributions with certain probabilities. The equation for the GMM is shown in equation (6) [1, p. 9].

$$p(x | \omega_i) = \sum_{k=1}^{M_i} c_{ik} \mathcal{N}(\mu_{ik}, \Sigma_{ik}) = \sum_{k=1}^{M_i} \frac{c_{ik}}{\sqrt{(2\pi)^D |\Sigma_{ik}|}} \exp \left[-\frac{1}{2} (x - \mu_{ik})^\top \Sigma_{ik}^{-1} (x - \mu_{ik}) \right] \quad (6)$$

For this model do we also need to calculate weights c_{ki} for each of the Gaussian in each class ω_i . The weights must sum up to a total of one [1, p. 9].

2.3 Model-fitting and generalization

When training a classifier, the goal is not only to make accurate predictions on the training data, but more importantly, to generalize well to unseen data. A well-fitted model should be able to capture the underlying patterns in the data without simply memorizing the training examples. This ability to generalize is what makes the model useful in real-world applications.

There are two common pitfalls when fitting a model, underfitting and overfitting [2, p. 20-31].

Underfitting occurs when the model is too simple to capture the structure of the data. It may result from using an overly basic model or insufficient training data, leading to high error both on the training set and on new data.

On the other hand, overfitting happens when the model is too complex relative to the data, and ends up learning noise or specific data points in the training set. Such a model performs very well on training data, but poorly on unseen test data, as it fails to generalize. Figure 1 shows the concept of underfitting, fitting and overfitting of a model.

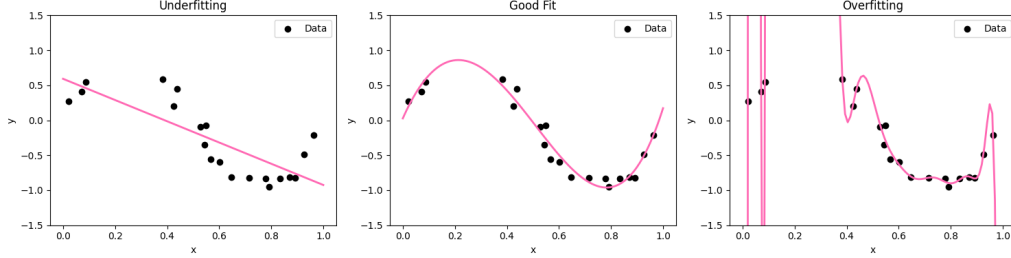


Figure 1: Visual model showing, underfitting, fitting and overfitting

Evaluating model performance using a confusion matrix or error rates can help see how the model is fitted.

2.4 Evaluation of classifiers

Evaluating the performance of a classifier is an important part of any classification task. One useful tool for this purpose is the confusion matrix, which gives insight into how well the classifier distinguishes between different classes.

A confusion matrix is a table that compares the true class labels with the predicted class labels [1, p. 31]. Each row of the matrix represents the actual class, while each column represents the predicted class. The diagonal entries show the number of correctly classified samples, whereas the off-diagonal entries show misclassifications, that is, where the classifier confused one class for another. The general form for a three-class problem is shown in Table 1, where each entry a_{ij} represents the number of samples of true class i that were predicted as class j .

Table 1: Confusion matrix (theoretical example with three classes)

True \ Predicted	Class 1	Class 2	Class 3
Class 1	a_{11}	a_{12}	a_{13}
Class 2	a_{21}	a_{22}	a_{23}
Class 3	a_{31}	a_{32}	a_{33}

From the table, we can then calculate the error rate of each case, training and testing as shown in equation (7).

$$\text{Error rate} = \frac{\text{Misclassifications}}{\text{Total samples used}} \quad (7)$$

This matrix not only gives the total number of correct and incorrect classifications, but also allows us to see which specific classes are being confused with each other. This can be especially useful in multi-class classification problems, where some classes may be more similar and therefore more difficult to separate than others. This kind of analysis helps identify which classes are most frequently misclassified.

3 Our task

This section describes the tasks we have chosen, which forms the background for this report.

3.1 Iris task

The Iris dataset contains 150 samples evenly distributed among the three classes: Setosa, Versicolor, and Virginica. Each sample is described by four features: sepal length, sepal width, petal length, and petal width. The classification task is structured in two parts.

Task 1 involves training a linear classifier using two predefined data splits to evaluate its ability to generalize. In the first split, the first 30 samples from each class are assigned to the training set, and the remaining 20 to the test set. In the second split, the last 30 samples per class are used for training, and the first 20 for testing. Task 2 focuses on the evaluation of linear separability across the features. Histograms for each feature and class are used to assess the degree of class overlap. The feature showing the most overlap is excluded one by one. Classifiers are then trained using three, two, and one feature(s), respectively, to analyze how feature selection influences classification performance.

3.2 Pronounced vowels task

The dataset consists of recordings from 139 individuals pronouncing 12 distinct vowels. Features are extracted from the recordings based on peaks in the frequency spectrum. The classification is based on the first three features, using the first 70 samples for training and the remaining 69 samples for testing. The data is non-linearly separable due to overlapping frequency peaks across classes.

The task is divided into two parts. The first part involves using a single Gaussian model per class, with both full and diagonal covariance matrices, to perform vowel classification. The resulting confusion matrices and error rates for each covariance type are compared to assess model performance. The second part applies Gaussian mixture models (GMMs) using two and three components per class, with diagonal covariance matrices. Confusion matrices and error rates are computed for both GMM configurations. Lastly, all four model types are then compared to identify which vowel classes exhibit the most significant performance differences.

4 Implementation and results

In this section there will be a thorough guide of the implementation and results of the Iris flower classification in section 4.1. Then the implementation and results of the vowels classification problem will be presented in section 4.2. Both classifiers were implemented in Python as it was the preferred programming language. The implementation relied on NumPy [3], along with supporting libraries such as Pandas [4], Scikit-learn [5], Seaborn [6], and Matplotlib [7] for data handling and visualization. All the code used to implement and test is available on GitHub [8].

4.1 Iris task

This section describes how the linear classifier was implemented and trained using the Iris dataset.

The Iris dataset consisted of 150 samples where each of the three classes: Iris-setosa, Iris-versicolor, and Iris-virginica contain 50 samples each. Each sample is described by four features: sepal length, sepal width, petal length, and petal width. To prepare the data for analysis, the

dataset was loaded into a Pandas DataFrame and the class labels were mapped with the numerical values 0, 1, and 2 for use in the classification models. To separate training and test data, the task specification was followed and two instances were tested. First, the first 30 samples of each class were selected for training and the last 20 for testing. This results in 90 training samples and 60 test samples. For the second instance, the last 30 samples were selected for the training set and the first 20 samples of each class was used for the testing set.

The classifier uses a sigmoid function defined by equation (3). This function transforms the output of the linear model into a value between 0 and 1, which can be interpreted as the probability that the input belongs to a given class. Each target label was one-hot encoded before training, meaning they were converted into a vector with a 1 at the correct class index and 0 s elsewhere. This was necessary because the MSE cost function in equation (1) assumes that both the predictions, g , and the targets, t , are vectors of length C . The output g , defined in equation (2), gives out class probabilities. One-hot encoding allows MSE to compare these predicted probabilities with the correct class for each sample.

The linear classifier was trained using gradient descent to minimize the mean squared error (MSE). The feature vectors were augmented with a constant bias term, and the weight matrix was initialized to zeros. Updates were carried out iteratively through fully vectorized batch gradient descent, eliminating explicit loops for computational efficiency.

An experiment was conducted to select an appropriate learning rate, α . Figure 2 illustrates the convergence behavior of the MSE for different learning rates.

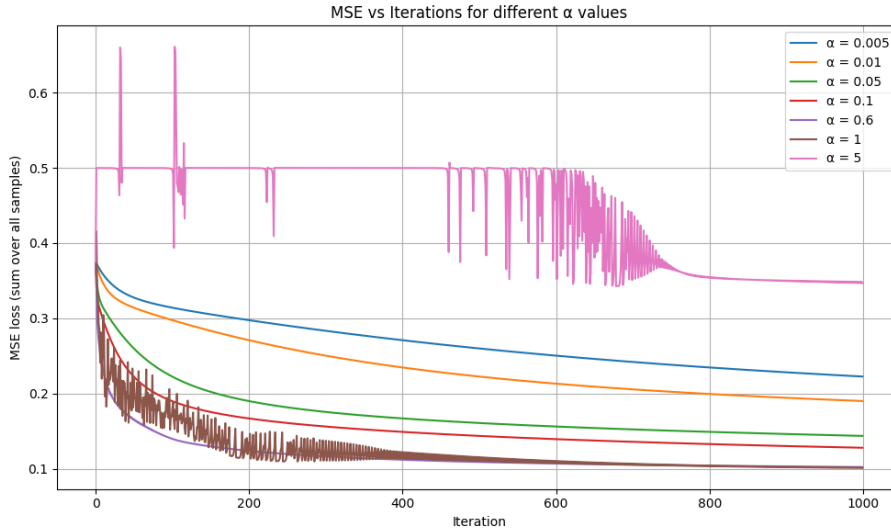


Figure 2: MSE versus iterations for different values of the learning rate α . The purple curve, corresponding to $\alpha = 0.6$, performs best with the lowest MSE across iterations.

From the results of the experiment, a relatively high learning rate of $\alpha = 0.6$ was chosen because the fully vectorized implementation made larger gradient updates necessary to ensure efficient convergence. Lower learning rates resulted in significantly slower convergence, while excessively high rates, $\alpha > 1$ led to instability and divergence. $\alpha = 0.6$, offered an optimal balance between speed and stability. The final model was trained for 2000 iterations resulting in stable convergence and minimal classification error.

After training, predictions were made by selecting the class with the highest probability. The final weights, W , were those that minimized the mean squared error (MSE) during training. Performance was evaluated using confusion matrices as shown in figures 3 and 4, and error rates as

per equation (7) on both the training- and test sets.

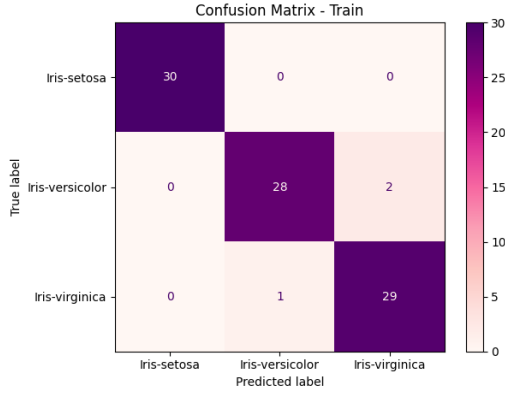


Figure 3: Confusion matrix for the training data using the first 30 samples for training and last 20 for testing. True classes are on the vertical axis; predicted classes on the horizontal.

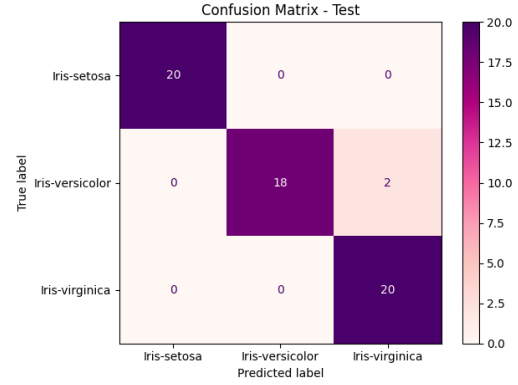


Figure 4: Confusion matrix for the testing data using the first 30 samples for training and last 20 for testing. True classes are on the vertical axis; predicted classes on the horizontal.

Figures 3 and 4 show the confusion matrices for case 1, where the model was trained on the first 30 samples. The training and test error rates were identical at 3.33%.

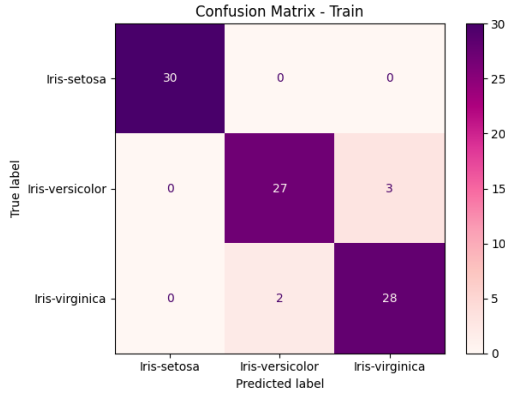


Figure 5: Confusion matrix for the training data using the last 30 samples for training and first 20 for testing. True classes are on the vertical axis; predicted classes on the horizontal.

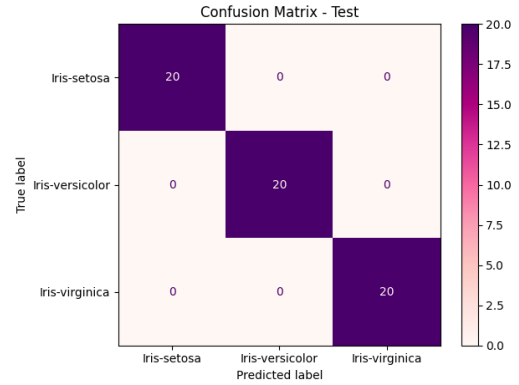


Figure 6: Confusion matrix for the testing data using the last 30 samples for training and first 20 for testing. True classes are on the vertical axis; predicted classes on the horizontal.

Figure 6 shows the confusion matrices for training and testing in case 2. The training error was 5.56%, slightly higher than in case 1, while the test error was 0.00%, indicating perfect classification on the test set. The confusion matrices show that class 0 was always correctly identified, while misclassifications occurred between classes 1 and 2.

To explore the contribution of each feature to linear separability as required in part 2 of the task, class-wise histograms were generated for each feature as shown in figure 7.

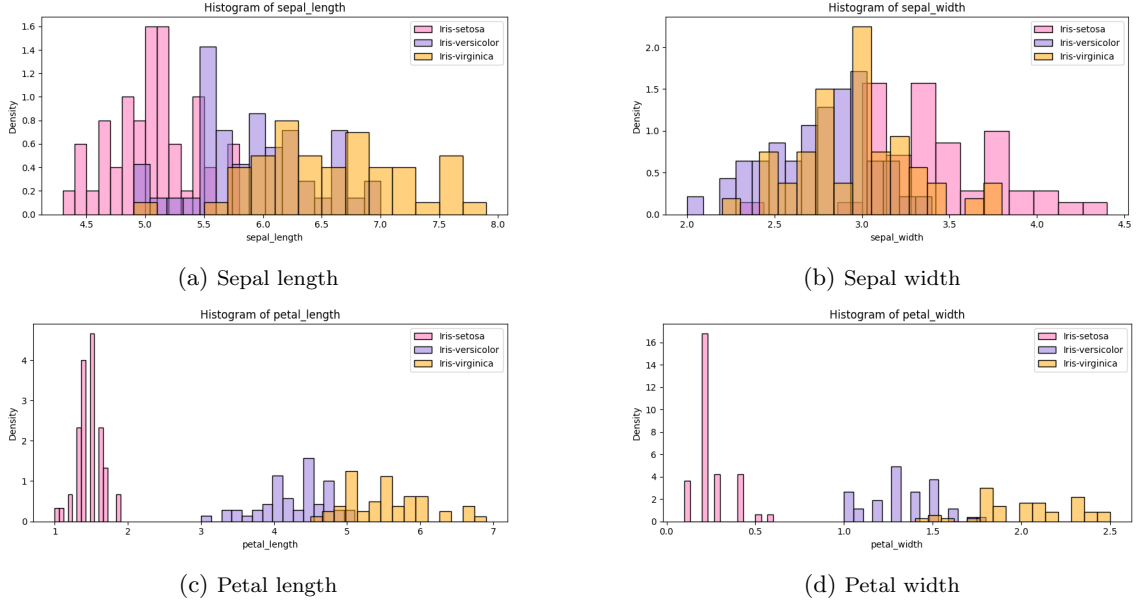


Figure 7: Histograms showing feature distributions across the classes; iris-setosa (pink), iris-versicolor (purple) and iris-virginica (orange).

The training and evaluation steps were repeated using case 1, where the first 30 samples were used for training and the last 20 for testing. Subsets of features (three, two, and one) were used by removing those with the most overlap and observed how performance deteriorated as features were removed. This allowed us to analyze which features contribute most to the classifier’s ability to separate the classes linearly. The features that had the most overlap were determined by looking at the histograms shown in figure 7. Figure 13 in the appendix A shows the confusion matrices for every test performed.

The results of the classification experiments are summarized in table 2. It compares training and test error rates across different settings, highlighting the impact of both data splits and feature selection. Experiments 1 and 2 relate to task 1, where the entire set of features is used with different training/test splits. Experiments 3 to 5 correspond to task 2, where subsets of features are used to evaluate their influence on classification performance. The table indicates which features were included in each set-up and whether the first 30 samples were used for training. Here, SW and SL refer to sepal width and sepal length, respectively, while PW and PL refer to petal width and petal length.

Table 2: Classification error for different feature subsets and data splits

Experiment	First 30 samples	SW	SL	PL	PW	Training Error [%]	Test Error [%]
1	x	x	x	x	x	3,33	3,33
2		x	x	x	x	5,56	0,00
3	x		x	x	x	3,33	5,00
4	x			x	x	6,67	6,67
5	x				x	12,22	8,33

4.2 Pronounced vowels task

In this subsection, the implementation and results for classifying pronounced vowels task, described in subsection 3.2, will be discussed. The theory about plug-in-MAP classifiers, explained in section 2.2, will be used.

The process began by extracting the names of all vowels and features that was needed and storing them in a dictionary. The dataset was then split, with 70 samples allocated for training and 69 samples for testing.

The first classifier was implemented by using a single Gaussian distribution. When modeling the data with a Gaussian distribution, theory indicates that the sample mean μ_i and sample covariance matrices Σ_i must be determined. These were computed using the *numpy.mean* and *numpy.cov* functions from the NumPy library [3]. Diagonal covariance matrices were implemented by multiplying the full covariance matrices with the identity matrix. Then the equation for single gaussian density, shown in equation (5), was used to compute the likelihood for a sample x_k belonging to each class ω_i , and the class ω_i with the highest posterior likelihood $p(x|\omega_i)$ was selected.

To evaluate the classifier, confusion matrices and error rates were calculated for both full and diagonal covariance matrices. The error rate for the full covariance matrices was 34.18% and the confusion matrix are shown in figure 8. The error rate with the diagonal covariance matrices was 53.86% and the confusion matrix are shown in figure 9.

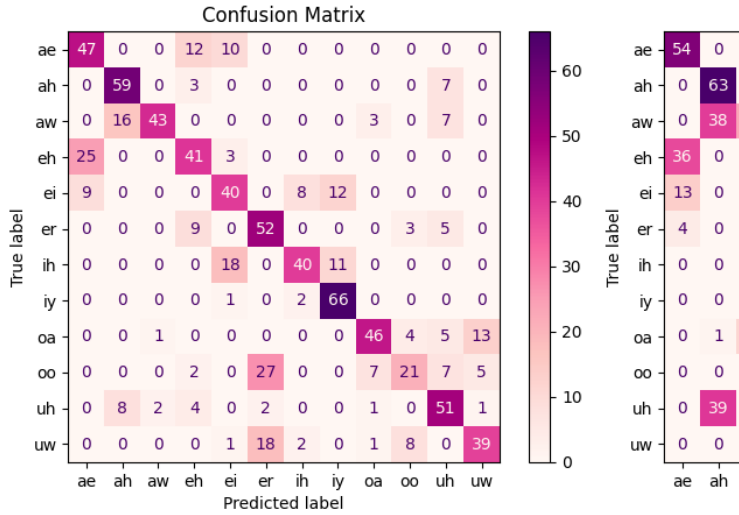


Figure 8: Confusion matrix for the test data using a single Gaussian classifier with full covariance matrices. The true vowels are represented on the vertical axis, while the predicted vowels are displayed on the horizontal axis.

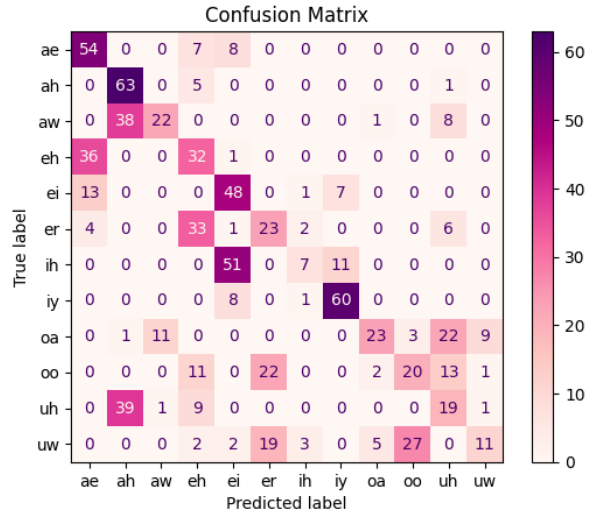


Figure 9: Confusion matrix for the test data using a single Gaussian classifier with diagonal covariance matrices. The true vowels are represented on the vertical axis, while the predicted vowels are displayed on the horizontal axis.

We can observe from the confusion matrix and the error rates that the classifier with full covariance matrices have a much higher accuracy than with diagonal covariance matrices. The classification performance varies across different classes. For example, "oo" is misclassified as "er" more often than it is correctly identified. Similarly, "ih" is classified as "ei" 51 times, but correctly identified only 7 times in the case with diagonal covariances. In contrast, some classes, such as "iy," are classified correctly almost every time.

Next, GMMs with diagonal covariance matrices were implemented for both two and three mixtures per class. To make GMM for each vowel, the built in function *sklearn.mixture.GaussianMixture* from the sklearn library were used [5]. This function takes as input a matrix containing all training samples from a given class, along with the desired number of Gaussian components to be used in the mixture. It then estimates the parameters of the mixture model, including the means and covariance matrices for each Gaussian component, as well as the mixing weights. Then, we use the built-in fit function to train the model, selecting the parameter set that gives the highest

likelihood for the vowel. Then, the function `score_samples` is used to estimate the likelihood for a sample belongs to each vocal. The classification process then followed the same approach as the single Gaussian model, where each sample was assigned to the class with the highest a posteriori likelihood. Then the error rates and confusion matrices were calculated. For GMM-2 is the error rate 37.08% and the confusion matrix is shown in figure 10. For GMM-3 is the error rate 36.26% and the confusion matrix is shown in figure 11.

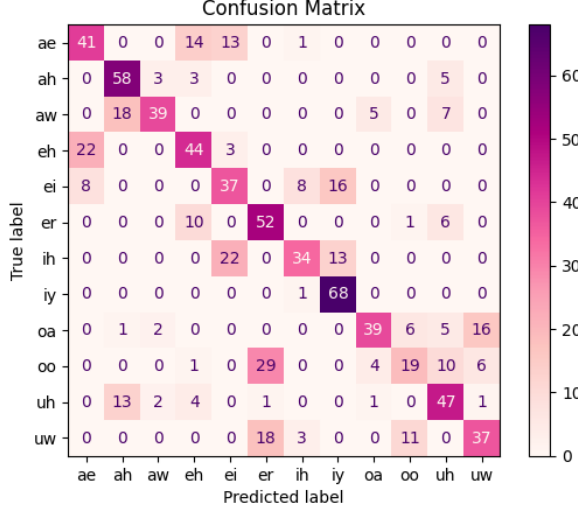


Figure 10: Confusion matrix for the test data using GMM-2 classifier with diagonal covariance matrices. The true vowels are represented on the vertical axis, while the predicted vowels are displayed on the horizontal axis.

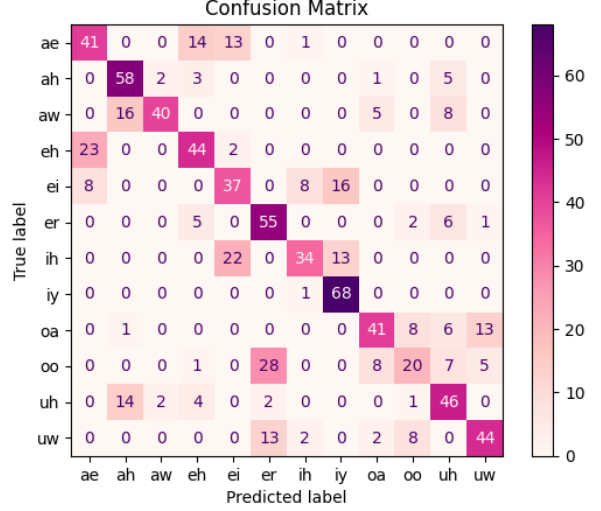


Figure 11: Confusion matrix for the test data using a GMM-3 classifier with diagonal covariance matrices. The true vowels are represented on the vertical axis, while the predicted vowels are displayed on the horizontal axis.

The error rate for GMM-3 is slightly better than GMM-2, but both is better than for the single gaussian distribution with diagonal covariance matrices. The classification of "ih", "uh" and "uw" is much better for this new distributions. Both GMM-2 and GMM-3 is still worse than a single distribution with full covariance matrices.

5 Discussion

5.1 Classification of Iris flowers

This section discusses the results from Section 4.1 and Table 2, focusing on what the experiments reveal about the model's performance, generalization, and the role of features in linear classification.

In the first task, we tested how the model generalizes to unseen data using two different training/test splits. Both splits gave low training error, but the test error was lower in the second case. This may be because the samples used for testing in case 2 were more easily separated, especially between the Versicolor and Virginica classes. However, since the dataset is small, such differences could simply reflect the variance in how the data was split. This shows how sensitive simple models can be to the training-test division. A better way to evaluate generalization would be to use k-fold cross-validation, which tests the model on several different splits to get a more reliable result. However, the small gap between training and test error suggests that the model generalizes well in this setting, and we see no signs of overfitting.

The second task explored how removing features affects performance. As features were removed, the classification accuracy dropped. This shows that petal length and petal width are the most

important features, especially for distinguishing Versicolor from Virginica. In contrast, sepal-based features contributed less as there was a lot of overlap. When only petal width was used, the model still managed to classify Setosa correctly as Setosa is clearly separated from the other classes in that dimension. These results match what we saw in the histograms, where the petal features showed clear separation between classes, while the sepal features overlapped more.

Removing features reduces the amount of information the model has available, and with fewer dimensions to work with, the model begins to underfit. Since a linear classifier can only draw straight boundaries, it needs enough informative features to place those boundaries correctly. As we remove useful features, bias increases and the model loses its ability to distinguish between overlapping classes. On the other hand, no signs of overfitting were observed.

These experiments show how important good features and evaluation methods are when using simple models. The Iris dataset works well with linear classification because of its structure, but more complex data with noise or non-linear patterns would likely require more flexible models.

5.2 Classification of pronounced vowels

In this subsection we will discuss the results of the classification of pronounced vowels in subsection 4.2.

We observe a clear diagonal in all the confusion matrices, indicating that many predictions are correct. However, there are some darker regions outside the diagonal and the overall error rates are still relatively high, all above 30%. To investigate possible reasons for this, we looked more closely at the dataset split. In this task, we used the first 70 samples for training and the last 69 for testing. By examining the dataset, we noticed that the training set consists only of men and women, while the test set contains only a few women and the rest children. Since we are generally able to distinguish between the voices of children, women, and men, due to differences in pitch, formant frequencies, and vocal tract length, this imbalance may have a significant effect on model performance. To evaluate the impact of this imbalance, we plotted the vowels "oo" and "er" for both the training and test sets. The plots are shown in figure 12.

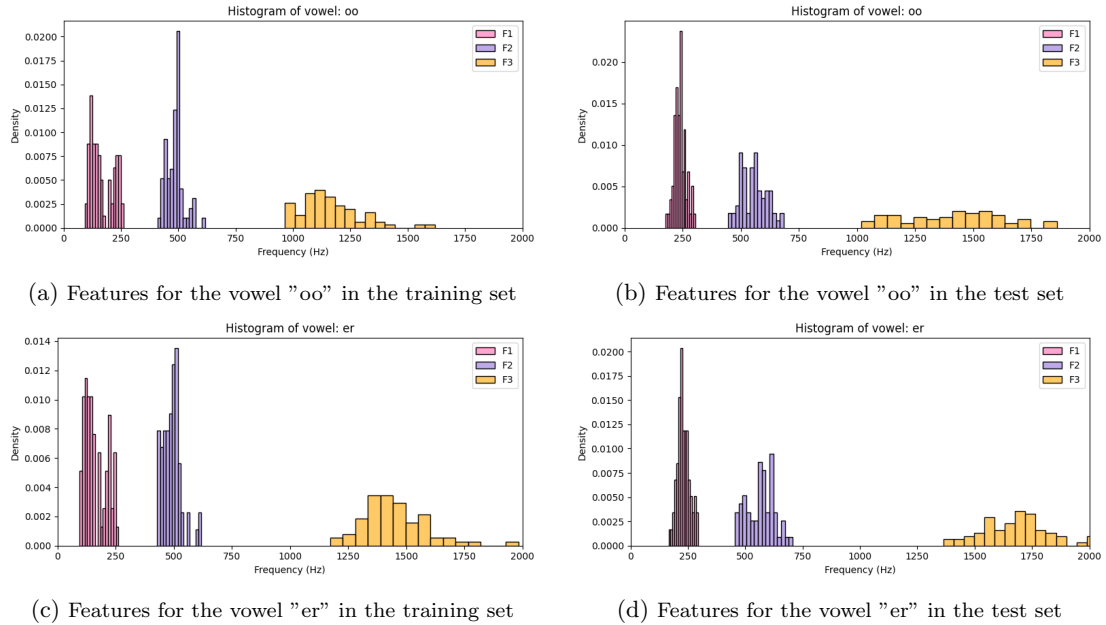


Figure 12: Histogram of features F1 (pink), F2 (purple), and F3 (yellow) for vowels "oo" and "er" in both the training and test sets.

Confusion matrices showed that "oo" was often predicted as "er" instead of correctly. Looking at the features in figure 13, we can see that the first two, $F1$ and $F2$, are quite similar, making it hard to tell them apart. The main reason "oo" gets misclassified more often than "er" is that the third feature $F3$ for "oo" in the test set has a high variance and fits the distribution of $F3$ for "er" better than for "oo" for the training set. By mixing the data to ensure the model trains on all types of people, it will likely perform better. Another way of improving the classifiers is by increasing the training data, but this will also extend the run time.

In section 4.2 we observed that the classifier using full covariance matrices generally produced more accurate predictions than the one using diagonal covariance matrices. The diagonal assumption implies that the covariance between different features is zero, which is equivalent to assuming linear independence between them. However, our results indicate that this simplification gives significantly worse results.

The GMM-3 classifier achieved the lowest error rate among the three models using diagonal covariance matrices. However, the best overall performance was still achieved by the classifier based on a single Gaussian distribution with full covariance matrices. The significant improvement of GMM-3 over the single Gaussian model with diagonal covariance indicates that the true underlying distribution has multiple peaks. Given that the full covariance model performed better than the diagonal, and GMM-3 was the best among the diagonal covariance models, it is reasonable to assume that a GMM-3 with full covariance matrices would have given the best results.

6 Conclusion

This project evaluated classification performance on two datasets using different modeling approaches. For the Iris dataset, a linear classifier trained with gradient descent and mean squared error (MSE) achieved good results. The best performance was achieved when training on the last 30 samples of each class, resulting in a test error of 0.00%. When training on the first 30 samples, the test error was slightly higher at 3.33%. Feature analysis showed that petal length and petal width were the most informative features. Removing these led to a gradual increase in test error to 6.67% with two features and 8.33% with one. Overall, the project confirms that a linear classifier is effective for linearly separable data when trained with informative features, but its performance is sensitive to both data splitting and feature selection.

We implemented and evaluated a classifier based on a single Gaussian distribution using both full and diagonal covariance matrices. The error rate was 34.18% for full covariance matrices and 53.86% for diagonal, indicating that the features are not linearly independent. We then extended our approach by implementing classifiers using Gaussian Mixture Models (GMM) with two and three components, both with diagonal covariance matrices. The error rate for GMM-3 was 36.26%, slightly better than GMM-2, which had an error rate of 37.08%. Comparing all classifiers, we found that GMM-3 was the best among those using diagonal covariance, but the single Gaussian model with full covariance performed the best overall. This suggests that GMM-3 with full covariance matrices would likely have achieved the best results.

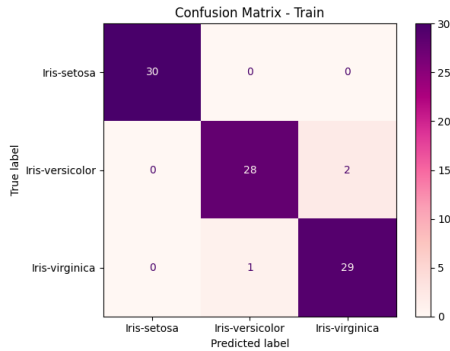
The classification works quite well, as we see a clear diagonal line in the confusion matrix. However, we have also identified several areas for improvement. The training set contained 70 samples of only men and women, while the test set had 69 samples of only women and children. A more balanced dataset with different types of people in both sets would likely improve the results. However, increasing the sample size also extends runtime, so this trade-off must be considered.

Bibliography

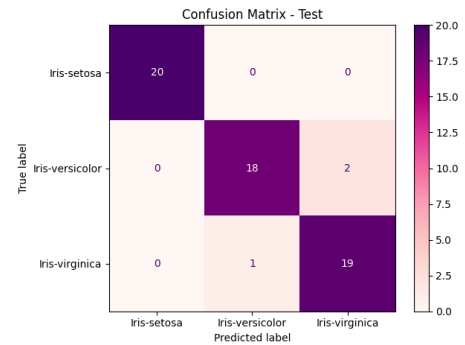
- [1] Magne H. Johnsen. ‘Classification’. In: *TTT4275 Estimering, deteksjon og klassifisering (2025 VÅR)* (2018).
- [2] Richard O. Duda, Peter E. Hart and David G. Stork. *Pattern Classification*. 2nd ed. New York: Wiley-Interscience, 2001.
- [3] Charles R. Harris et al. ‘Array programming with NumPy’. In: *Nature* 585.7825 (Sept. 2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2. URL: <https://doi.org/10.1038/s41586-020-2649-2>.
- [4] Wes McKinney. ‘Data Structures for Statistical Computing in Python’. In: *Proceedings of the 9th Python in Science Conference* (2010). Ed. by Stéfan van der Walt and Jarrod Millman, pp. 51–56. DOI: 10.25080/Majora-92bf1922-00a.
- [5] Scikit-learn. *GaussianMixture*. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.mixture.GaussianMixture.html#sklearn.mixture.GaussianMixture.fit> (visited on 29th Mar. 2025).
- [6] Michael L. Waskom. ‘Seaborn: Statistical Data Visualization’. In: *Journal of Open Source Software* 6.60 (2021), p. 3021. DOI: 10.21105/joss.03021.
- [7] J. D. Hunter. ‘Matplotlib: A 2D Graphics Environment’. In: *Computing in Science & Engineering* 9.3 (2007), pp. 90–95. DOI: 10.1109/MCSE.2007.55.
- [8] Karin Skinderhaug Arta Osmani. *TTT4275 - Classification Project*. <https://github.com/Artaosmani/TTT4275-classification-project>. 2025.

Appendix

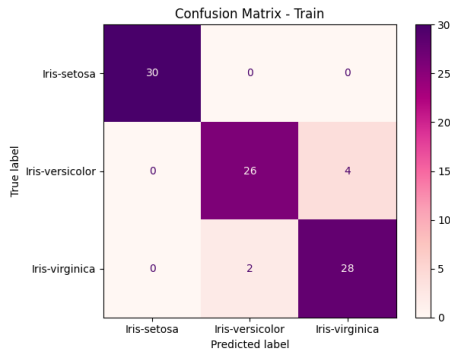
A Confusion matrix task 2



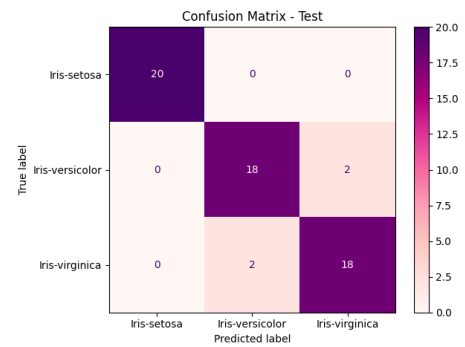
(a) Confusion matrix from the training set with $\alpha = 0.6$ and an error rate of 3.33%. Removed the feature *Sepal width*.



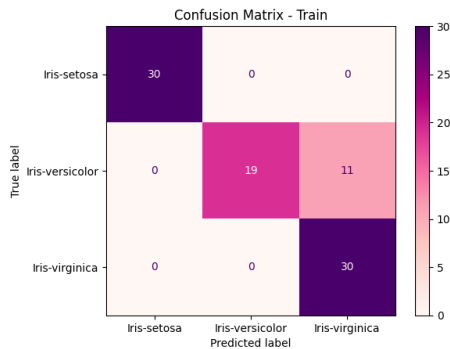
(b) Confusion matrix from the test set with $\alpha = 0.6$ and an error rate of 5.00%. Removed the feature *Sepal width*.



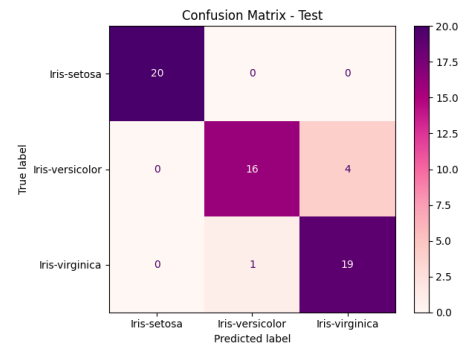
(c) Confusion matrix from the train set with $\alpha = 0.6$ and an error rate of 6.67%. Removed the features *Sepal width* and *Sepal length*.



(d) Confusion matrix from the test set with $\alpha = 0.6$ and an error rate of 6.67%. Removed the features *Sepal width* and *Sepal length*.



(e) Confusion matrix from the training set with $\alpha = 0.6$ and an error rate of 12.22%. Removed the features *Sepal width* and *Sepal length* and *Petal length*.



(f) Confusion matrix from the test set with $\alpha = 0.6$ and an error rate of 8.33%. Removed the features *Sepal width*, *Sepal length* and *Petal length*.

Figure 13: Confusion matrices for training and test including different amount of features